

Binary

二分

本页面将简要介绍二分查找，由二分法衍生的三分法以及二分答案。

二分法

定义

二分查找（英语：binary search），也称折半搜索（英语：half-interval search）、对数搜索（英语：logarithmic search），是用来在一个有序数组中查找某一元素的算法。

过程

以在一个升序数组中查找一个数为例。

它每次考察数组当前部分的中间元素，如果中间元素刚好是要找的，就结束搜索过程；如果中间元素小于所查找的值，那么左侧的只会更小，不会有所查找的元素，只需到右侧查找；如果中间元素大于所查找的值同理，只需到左侧查找。

性质

时间复杂度

二分查找的最优时间复杂度为 $O(\log n)$ 。

二分查找的平均时间复杂度和最坏时间复杂度均为 $O(\log n)$ 。因为在二分搜索过程中，算法每次都把查询的区间减半，所以对于一个长度为 n 的数组，至多会进行 $\log_2 n$ 次查找。

空间复杂度

迭代版本的二分查找的空间复杂度为 $O(1)$ 。

递归（无尾调用消除）版本的二分查找的空间复杂度为 $O(\log n)$ 。

实现

```
1 int binary_search(int start, int end, int key) {
2     int ret = -1; // 未搜索到数据返回-1下标
3     int mid;
4     while (start <= end) {
5         mid = start + ((end - start) >> 1); // 直接平均可能会溢出，所以用这个算法
6         if (arr[mid] < key)
7             start = mid + 1;
8         else if (arr[mid] > key)
9             end = mid - 1;
10        else { // 最后检测相等是因为多数搜索情况不是大于就是小于
11            ret = mid;
12            break;
13        }
14    }
15    return ret; // 单一出口
16 }
```

▼ Note

参考 [编译优化 #位运算代替乘法](#)，对于 是有符号数的情况，当你可以保证 时， $n >> 1$ 比 $n / 2$ 指令数更少。

最大值最小化

注意，这里的有序是广义的有序，如果一个数组中的左侧或者右侧都满足某一种条件，而另一侧都不满足这种条件，也可以看作是一种有序（如果把满足条件看做，不满足看做，至少对于这个条件的这一维度是有序的）。换言之，二分搜索法可以用来查找满足某种条件的最大（最小）的值。

要求满足某种条件的最大值的最小可能情况（最大值最小化），首先的想法是从小到大枚举这个作为答案的「最大值」，然后去判断是否合法。若答案单调，就可以使用二分搜索法来更快地找到答案。因此，要想使用二分搜索法来解决这种「最大值最小化」的题目，需要满足以下三个条件：

1. 答案在一个固定区间内；
2. 可能查找一个符合条件的值不是很容易，但是要求能比较容易地判断某个值是否是符合条件的；
3. 可行解对于区间满足一定的单调性。换言之，如果 是符合条件的，那么有 或者 也符合条件。（这样下来就满足了上面提到的单调性）

当然，最小值最大化是同理的。

STL 的二分查找

C++ 标准库中实现了查找首个不小于给定值的元素的函数 [std::lower_bound](#) 和查找首个大于给定值的元素的函数 [std::upper_bound](#)，二者均定义于头文件 `<algorithm>` 中。

二者均采用二分实现，所以调用前必须保证元素有序。

bsearch

bsearch 函数为 C 标准库实现的二分查找，定义在 `<stdlib.h>` 中。在 C++ 标准库里，该函数定义在 `<cstdlib>` 中。qsort 和 bsearch 是 C 语言中唯二的两个算法类函数。

bsearch 函数相比 qsort（[排序相关 STL](#)）的四个参数，在最左边增加了参数「待查元素的地址」。之所以按照地址的形式传入，是为了方便直接套用与 qsort 相同的比较函数，从而实现排序后的立即查找。因此这个参数不能直接传入具体值，而是要先将待查值用一个变量存储，再传入该变量地址。

于是 bsearch 函数总共有五个参数：待查元素的地址、数组名、元素个数、元素大小、比较规则。比较规则仍然通过指定比较函数实现，详见 [排序相关 STL](#)。

bsearch 函数的返回值是查找到的元素的地址，该地址为 void 类型。

注意：bsearch 与上文的 lower_bound 和 upper_bound 有两点不同：

- 当符合条件的元素有重复多个的时候，会返回执行二分查找时第一个符合条件的元素，从而这个元素可能位于重复多个元素的中间部分。
- 当查找不到相应的元素时，会返回 NULL。

用 lower_bound 可以实现与 bsearch 完全相同的功能，所以可以使用 bsearch 通过的题目，直接改写成 lower_bound 同样可以实现。但是鉴于上述不同之处的第二点，例如，在序列 1、2、4、5、6 中查找 3，bsearch 实现 lower_bound 的功能会变得困难。

利用 bsearch 实现 lower_bound 的功能比较困难，是否一定就不能实现？答案是否定的，存在比较 tricky 的技巧。借助编译器处理比较函数的特性：总是将第一个参数指向待查元素，将第二个参数指向待查数组中的元素，也可以用 bsearch 实现 lower_bound 和 upper_bound，如下文示例。只是，这要求待查数组必须是全局数组，从而可以直接传入首地址。

```

1 int A[100005]; // 示例全局数组
2
3 // 查找首个不小于待查元素的元素的地址
4 int lower(const void *p1, const void *p2) {
5     int *a = (int *)p1;
6     int *b = (int *)p2;
7     if ((b == A || compare(a, b - 1) > 0) && compare(a, b) > 0)
8         return 1;
9     else if (b != A && compare(a, b - 1) <= 0)
10        return -1; // 用到地址的减法, 因此必须指定元素类型
11    else
12        return 0;
13 }
14
15 // 查找首个大于待查元素的元素的地址
16 int upper(const void *p1, const void *p2) {
17     int *a = (int *)p1;
18     int *b = (int *)p2;
19     if ((b == A || compare(a, b - 1) >= 0) && compare(a, b) >= 0)
20        return 1;
21     else if (b != A && compare(a, b - 1) < 0)
22        return -1; // 用到地址的减法, 因此必须指定元素类型
23    else
24        return 0;
25 }

```

因为现在的 OI 选手很少写纯 C，并且此方法作用有限，所以不是重点。对于新手而言，建议老实地使用 C++ 中的 `lower_bound` 和 `upper_bound` 函数。

二分答案

解题的时候往往会考虑枚举答案然后检验枚举的值是否正确。若满足单调性，则满足使用二分法的条件。把这里的枚举换成二分，就变成了「二分答案」。

▼ [Luogu P1873 砍树](#)

伐木工人米尔科需要砍倒 米长的木材。这是一个对米尔科来说很容易的工作，因为他有一个漂亮的新伐木机，可以像野火一样砍倒森林。不过，米尔科只被允许砍倒单行树木。

米尔科的伐木机工作过程如下：米尔科设置一个高度参数（米），伐木机升起一个巨大的锯片到高度，并锯掉所有的树比 高的部分（当然，树木不高于 米的部分保持不变）。米尔科就得到树木被锯下的部分。

例如，如果一行树的高度分别为，米尔科把锯片升到 米的高度，切割后树木剩下的高度将是，而米尔科将从第 棵树得到 米木材，从第 棵树得到 米木材，共 米木材。

米尔科非常关注生态保护，所以他不会砍掉过多的木材。这正是他尽可能高地设定伐木机锯片的原因。你的任务是帮助米尔科找到伐木机锯片的最大的整数高度，使得他能得到木材至少为 米。即，如果再升高 米锯片，则他将得不到 米木材。

- 解题思路
- 参考代码

三分法

引入

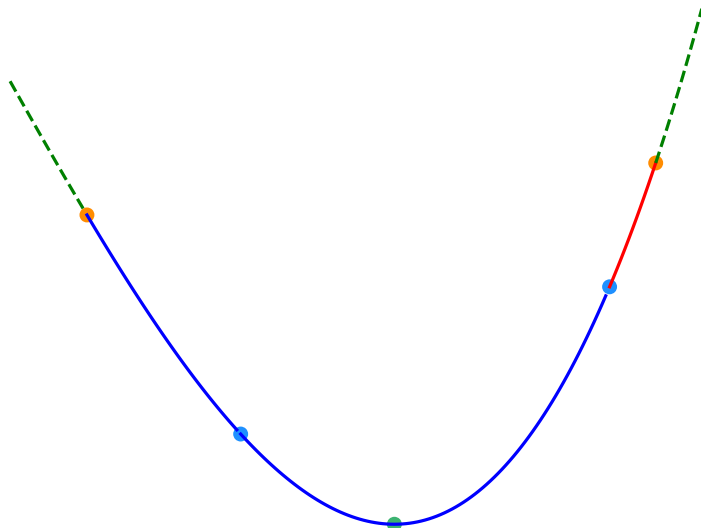
如果要求出单峰函数的极值点，通常使用二分法衍生出的三分法求单峰函数的极值点。

- 为什么不通过求导函数的零点来求极值点？

▼ 注意

只要函数是单峰函数，三分法既可以求出其最大值，也可以求出其最小值。为行文方便，除特殊说明外，下文中均以求单峰函数的最小值为例。

三分法与二分法的基本思想类似，但每次操作需在当前区间（下图中除去虚线范围内的部分）内任取两点（下图中的两蓝点）。如下图，如果 $f(lmid) > f(rmid)$ ，则在（下图中的红色部分）中函数必然单调递增，最小值所在点（下图中的绿点）必然不在这一区间内，可舍去这一区间。反之亦然。



▼ 注意

在计算 $lmid$ 和 $rmid$ 时，需要防止数据溢出的现象出现。

三分法每次操作会舍去两侧区间中的其中一个。为减少三分法的操作次数，应使两侧区间尽可能大。因此，每一次操作时的 $lmid$ 和 $rmid$ 分别取 $l + \frac{r-l}{3}$ 和 $r - \frac{r-l}{3}$ 是一个不错的选择。

实现

伪代码

C++

```
1 while (r - l > eps) {
2     mid = (l + r) / 2;
3     lmid = mid - eps;
4     rmid = mid + eps;
5     if (f(lmid) < f(rmid))
6         r = mid;
7     else
8         l = mid;
9 }
```

例题

▼ [洛谷 P3382 - 【模板】三分法](#)

给定一个 次函数和范围，求出使函数在 上单调递增且在 上单调递减的唯一的 的值。

- ▶ [解题思路](#)
- ▶ [参考代码](#)

习题

- [UVa 1476 - Error Curves](#)
- [UVa 10385 - Duathlon](#)
- [UOJ 162 - 【清华集训 2015】灯泡测试](#)
- [洛谷 P7579 - 「RdOI R2」称重 \(weigh\)](#)

分数规划

参见：[分数规划](#)

分数规划通常描述为下列问题：每个物品有两个属性，，要求通过某种方式选出若干个，使得 最大或最小。

经典的例子有最优比率环、最优比率生成树等等。

分数规划可以用二分法来解决。

本页面最近更新：2024/12/11 21:43:19, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者：[H-J-Granger](#), [NachtgeistW](#), [billchenchina](#), [CBW2007](#), [ChungZH](#), [countercurrent-time](#), [Enter-tainer](#), [FinParker](#), [flylai](#), [gavinliu266](#), [Great-designer](#), [HanwGeek](#), [Henry-ZHR](#), [HeRaNO](#), [i-yyi](#), [iamtwz](#), [inclyc](#), [lr1d](#), [ksyx](#), [LeiJinpeng](#), [leoleoasd](#), [Marcythm](#), [Selflocking](#), [shawlleyw](#), [shuzhouliu](#), [sshwy](#), [StudyingFather](#), [SukkaW](#), [Tiphereth-A](#), [TNO-C137](#), [Xeonacid](#), [yusancky](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用